

Runtime Safety in Move

Victor Gao, Wolfgang Grieskamp, Vineeth Kashyap, George Mitenkov,
Teng Zhang, Runtian Zhou, Andrea Cappa, and Marco Illadi

Aptos Labs, Palo Alto, USA

Abstract. Move is a smart-contract language originally developed for the Diem blockchain and now deployed at scale on Aptos. Move programs execute as fully typed bytecode, which lets the platform statically verify a small set of foundational safety properties — in particular type safety and reference safety — ahead of execution, during code loading time. While this design gives Move strong on-paper guarantees, the static verification logic is intricate, and historically it has been a source of bugs in the platform itself: subtle errors in the verifier can let bytecode through that violates the very rules the verifier is meant to enforce. In a smart-contract setting, such bugs are not academic: a missed type or reference rule violation can translate directly into loss of assets, forged authority, or unrecoverable corruption of on-chain state. For this reason, Aptos relies on a second layer of defense in the form of *runtime safety checks* that re-check the critical invariants during execution, providing protection against both compromised bytecode and latent verifier bugs. This paper describes the runtime safety checks in place for Move on Aptos.

Keywords: Move · Smart contracts · Runtime verification · Type safety · Reference safety.

1 Introduction

Move is a smart-contract language originally developed for the Libra/Diem blockchain at Meta [1, 4, 5] and now deployed at scale on Aptos [3], a high-throughput Layer-1 platform that settles transactions for hundreds of millions of dollars in user assets on chain. A defining trait of the language is its resource-oriented type discipline: digital assets are represented as Move *resources* whose ownership and movement are tracked by linear-style abilities (*copy*, *drop*, *store*, *key*), ruling out accidental duplication or destruction at compile time. The Aptos platform builds on this foundation with a system of objects, parallel execution, and a rich on-chain framework written in Move itself [2, 15].

Beyond its type discipline, Move has been designed from the outset to admit *offline formal verification*: the Move Prover translates Move programs and their specifications, written in the Move Specification Language, into verification conditions discharged by SMT solvers [6, 8]. These techniques are powerful but inherently offline: they apply prior to deployment, depend on hand-written specifications, and produce no signal once the code is running on chain.

Closer to the execution path, the Move VM also performs *static bytecode verification* every time a module is loaded, enforcing well-formedness, typing, and

Move’s reference discipline — a Rust-like [11, 13] aliasing and lifetime regime detailed in Section 2. The reference analysis in particular is intricate: the verifier maintains a borrow graph that conservatively approximates the aliasing relationships entailed by control- and data-flow. Implementing this correctly is a delicate engineering task, and the Aptos bug bounty program has surfaced verifier bugs in the past. In a security-oriented architecture, the cost of trusting a single intricate static analysis as the sole gate is too high.

The Aptos runtime therefore carries a second layer of *runtime safety checks* that re-establish the critical invariants during execution. The checks described in this paper fall into three groups. *Type checks* verify that the dynamic type of every value matches the static type the bytecode expects to consume. *Ability checks* verify that operations such as copy, drop, or storage in global state are only performed on values whose declared abilities permit them. *Reference checks* verify, at runtime, the same dangling- and aliasing-freedom properties that the static verifier is supposed to enforce.

Since these checks are costly, the runtime supports *trusted code* that waives them under certain conditions, and an *asynchronous* checking mode that leverages idle workers in Block-STM [7], Aptos’s speculative parallel executor.

[teng: I think we should explicitly state the contribution: 1) propose a second layer of in-depth-defense for blockchain which potentially holds millions of dollars, 2) present a set of dynamic checks that are necessary for the second layer to be effective, 3) present a mechanism for async check by leveraging block-STM, which is originally designed for throughput.]

2 Move on Aptos

We briefly summarize the aspects of Move on Aptos [3, 5] that the rest of the paper builds on.

- *Typing and generics.* Move is strongly statically typed. Generic functions and types are instantiated by substitution at the call site and tracked through the bytecode verifier. The substitution must reach every leaf expression, since accesses to type-indexed global storage (below) need the concrete instantiated type at runtime.
- *Abilities.* Each type carries a subset of the four abilities `copy`, `drop`, `store`, and `key`, which gate the corresponding operations. Types without `drop` or `copy` are linear: the compiler refuses to discard or duplicate them, and every value of such a type must be moved, stored, or explicitly destructured.
- *Algebraic data types.* Structs are nominal product types; enums are nominal sums whose variants carry payloads. Pattern matching may bind references directly into a variant’s fields.
- *References.* Immutable (`&T`) and mutable (`&mut T`) references obey a static aliasing discipline comparable to Rust’s [11, 13]: two live mutable references cannot point to overlapping memory, and references cannot outlive the values they borrow.
- *Global storage.* Global state is a map indexed by (resource type, address) pairs. Resources of types with the `key` ability are accessed through `T[addr]`,

$\&T[addr]$, and $\&mut\ T[addr]$, with the same linear-ownership discipline as for local values.

- *Bytecode and bytecode verification.* Move source compiles to a typed bytecode that the Move VM [4] executes. At module-load time, a bytecode verifier checks well-formedness, typing, abilities, and the reference discipline above. The runtime safety checks of this paper run during execution, after the verifier has accepted the bytecode.

3 Runtime Safety Checks

This section gives a conceptual overview of the runtime checks. Each of the checks is motivated by an example representing a severe security bug.

3.1 Type Safety

Move’s nominal typing means two structs with identical layout but different names are distinct types. A *type-confusion* exploit breaks that distinction, and is most damaging when one of the two types is privileged. Consider the following example:

```
module 0x1::admin {
  struct Capability(address);
  public fun admin_action(c: &Capability) {
    /* gated by Capability */
  }
}

module 0x2::attacker {
  struct FakedCapability(address);
}
```

The attacker freely constructs a `FakedCapability` (it owns the type) and only needs a route that delivers it to `admin_action` as if it were a `Capability`.

Example Back when Move was developed at Meta/Facebook, vector operations were added to the VM. The developer forgot to add a type check in the bytecode verifier for the vector element type on push. The compiler did perform this check, so no test case via the source language could reach the problem. However, it was possible to manually construct bytecode, skipping the compiler, which did the following (conceptually):

```
let v: vector<Capability> = vector::empty();
v.push_back(FakedCapability(victim)); // should be rejected
admin_action(&v[0]); // runs on a forged token
```

The two types share a layout, so every downstream read succeeds, and the gated action runs on a token its owner never minted. This bug was found as part of the Aptos bug bounty program before the network went to mainnet.

Performing the Checks In order to prevent type confusion, we perform type checks at runtime. To this end a type stack is maintained in parallel to the operand stack of the Move VM. Whenever a value is pushed on the operand stack, its type is also pushed on the type stack. This type is known since the

Move VM has typed registers. Whenever a value is popped from the stack and stored in a register or consumed by an operation, the corresponding type is popped as well and compared with the expected type, creating a runtime error if type confusion is detected.

3.2 Ability Safety

The most damaging ability bug is on `copy`. A value whose type lacks `copy` is meant to be unique: this is what makes the `Capability` of Section 3.1 a single-issue permission, and what makes a Coin balance impossible to counterfeit. A runtime that produces a fresh value of a `no-copy` type silently duplicates that resource.

Example While we do not know about a historical exploit around abilities, here is a simple imaginary example:

```
fun clone(c: &Capability): Capability {
    *c    // ReadRef requires 'copy' — should be rejected.
}
```

The `Capability` type introduced in Section 3.1 does not have the `copy` ability, thus the above code should be rejected. If for some reason the bytecode verifier does not do this, one can craft bytecode which silently creates a copy of the token.

Performing the Checks When executing untrusted code, the VM performs checks for violations of each of the abilities `copy`, `drop`, `key`, and `store`. Most of those checks are relatively straightforward and can be directly performed when the corresponding instruction is executed. The exception is `drop`, which requires the VM to track which values have not been consumed on function exit.

3.3 Reference Safety

The mutable-aliasing rule is critical in the presence of algebraic data types (enum type): a reference into a variant payload is only sound while the enum still occupies that variant. If two mutable references to the same enum can simultaneously be live, one can switch the variant under the other.

Example Consider the following code:

```
enum E { V1(Capability), V2(FakedCapability) }

fun aliased_swap(a: &mut E, b: &mut E, faked: &mut E) {
    // faked is V2(FakedCapability(victim))
    match (a) {
        E::V1(c) => {
            mem::swap(b, faked); // *a is now V2(..)
            admin_action(c);     // c still &Capability
        },
        _ => {},
    }
}
```

A call of the form `aliased_swap(&mut x, &mut x, ...)` should normally be rejected by the bytecode verifier’s borrow-graph analysis because `a` and `b` would alias.

This was a real issue discovered via the bug bounty program after the introduction of algebraic data types. There was a bug in the bytecode verifier’s borrow analysis which did not correctly identify borrow graph edges `e.f1` and `e.f2` from different variants to refer to the same physical field at the same offset. This allowed creating mutable references to the same field offset in the ADT, eventually enabling a call to `aliased_swap` with aliased mutable references.

Performing the Checks Performing reference safety for borrow semantics in the style of Move (or Rust) at execution time is a novel problem which, to the best of our knowledge, has not been previously attempted. The challenge is that during construction of references, mutable reference aliasing *is* allowed. Only once those references are used, aliasing must be forbidden. As an example consider the following code – obviously valid code which should be allowed:

```
struct S {a: u64, b: u64}
let s: S;
let r1 = &mut s.a;
let r2 = &mut s.b;
*r1 = 1; *r2 = 2;
```

In the bytecode we generate for this, this is actually broken down into:

```
borrow_loc s
borrow_field a
borrow_loc s    ← temporary aliasing
borrow_field b
```

Thus while we have a mutable reference to `s.a` on the stack, we create temporarily a new mutable reference to the whole of `s`, overlapping with the existing one. Only after we consume this reference and create `s.b`, aliasing has been eliminated. This property makes a naive approach to reference safety checking, for example via reference counting, infeasible.

Our implementation maintains a *shadow state* that runs in parallel to the operand stack and the frame stack of the VM. Every value carried by the running program is mirrored by a tag in the shadow stack: either “non-reference” or “reference ρ ”, where ρ is a fresh identifier minted at the borrowing site. For every non-reference value the shadow state can hold — a local, a globally borrowed resource, or the value sitting behind a reference parameter passed in by the caller — we build, lazily, an *access path tree*: a tree whose edges are field (or variant) offsets, and whose nodes record the set of references currently pointing at that sub-location. Vector elements collapse onto a single abstract edge so that vector-heavy programs do not blow up the tree. A reference ρ is then nothing more than a pointer into some access path tree, tagged with its mutability.

Aliasing is policed by *poisoning*. Whenever a destructive write happens through a mutable reference at some node n — this includes the implicit destructive write

a `move_loc` performs on the local it empties — the shadow state walks the tree around n and marks every *other* reference reachable from there as poisoned: all immutable references at n , its ancestors and its descendants, and all mutable references at strict descendants. The mutable reference doing the write itself survives, matching the way the static borrow checker accounts for the borrow. Any subsequent attempt to read, write, or further borrow through a poisoned reference aborts the transaction with an invariant-violation error.

At each function-call boundary the access path subtrees reached through the reference parameters are additionally locked — exclusively for mutable parameters, shared for immutable — which gives the callee the same single-writer / multiple-reader invariant the bytecode verifier statically assumes. References returned by the callee are required to be derived from one of its reference parameters and are translated back to the corresponding access path in the caller’s frame; native functions that mint or return references must declare an explicit reference model so the shadow state stays consistent across the call.

The bytecode fragment above runs unchanged: the second `borrow_loc s` creates a new mutable reference at the root, but it is consumed by `borrow_field b` before any destructive write fires, so no poisoning happens and execution proceeds. The variant-switch attack from the start of this section, in contrast, is caught: the call to `mem::swap` is a destructive write at the variant root, which is an ancestor of the still-live payload reference `c`, and so `c` is poisoned the moment `swap` returns — the subsequent `admin_action(c)` fails before it ever reads the forged token.

3.4 Reentrancy

Move on Aptos later gained *function values*: a closure that can be captured, passed around, stored (into the global storage), and invoked. This feature greatly enhanced the expressivity of the language (by enabling dynamic dispatch), but also reintroduced a classic challenge — *reentrancy*.

Prior to this, freedom from reentrancy was a *static* guarantee: a function declares its global resource accesses with the `acquires` annotations, and because module dependencies must be acyclic, no function a module calls can re-enter that module and borrow the same resource twice.

Here is an example demonstrating what a vulnerable pattern may look like, using a withdrawal function that processes the payout through a caller-supplied callback:

```
module 0x42::vault {
  struct Vault has key { balance: u64 }

  public fun withdraw(owner: address, payout: |u64|)
    acquires Vault {
      let amount = Vault[owner].balance;
      payout(amount);           // dynamic dispatch into
                                // unknown code
      Vault[owner].balance = 0;  // cleared only after the
                                // callback
```

```
    }
}
```

A malicious or poorly written `payout` could call `withdraw` again before the first withdrawal has been completed, resulting in the re-entrant call reading the same non-zero `balance`.

Runtime Reentrancy Check Our current solution to this problem is a runtime reentrancy check. The VM’s runtime maintains a map that for each module, records a count of how many times it has been entered. The count is incremented when entered from a different module, or from a function value, even if the underlying function is in the same module, and decremented on return.

A count greater than one is not in itself a violation. A transaction-aborting error is only fired when (1) a global resource defined by the reentered module is accessed, or (2) a module is reentered while previously locked by a *module lock*, which is a stricter mode activated by entering a function with the `#[module_lock]` attribute.

Unlike the type, ability and reference checks, which are redundant safety nets that reassert key properties guaranteed by the bytecode verifier, the reentrancy check is a *required* check, crucial for ensuring safety with dynamic dispatch. Its implementation however, shares the same spirit as the other runtime checks.

Because the reentrancy check is invoked on every function entry and return, it sits on the interpreter’s hot path. The cost is negligible today thanks to two optimizations (fast, non-cryptographic hash + module id interning), but could be further under test with our next-gen VM, which will have much faster function calls, making the per-call overhead more noticeable.

We are also exploring whether the check could instead be deferred to the asynchronous phase (Section 5). However its soundness requires more study as aborts caused by it are part of a transaction’s execution semantics, rather than a redundant post-execution validation.

3.5 Liveness

A fourth class of exploits targets liveness: an attacker submits a transaction that every validator processes the same way, and that processing crashes the node with out of memory. On Aptos, the most direct source of such a denial-of-service attack has historically been missing bounds on types, and here specifically, types which are created by *type substitution*. The Move runtime needs to know instantiated types in order to implement type-indexed resource access; thus even though Move is statically typed, types cannot be erased. A generic type `Pair<T, T>` instantiated with `T = Pair<U, U>` describes a tree whose representation doubles at every level of nesting. Without a bound on instantiation depth, a script that constructs `Pair<Pair<...>>` at nesting depth n forces the runtime to materialize type-information records — needed for type tagging, dispatch, and serialization — whose size grows as $O(2^n)$. At $n \approx 30$ the per-transaction allocator is at gigabytes; every validator OOMs in lockstep, and the chain halts.

The corresponding safety check is a runtime bound on type depth and aggregate type size, evaluated before any value of the offending type is constructed — a check the offline verifier cannot perform, because the type only comes into existence when the transaction is executed.

4 Trusted Code

Runtime safety checks are costly. The MoveVM supports the concept of *trusted code* which waives those checks. Currently, trusted code deployment underlies Aptos Foundation governance, and is only allowed for the Aptos framework, whose deployment needs to be approved by the Aptos operators. We are also investigating models for trusted code based on stake.

The main property of trusted code is that the governance model gives higher confidence in the integrity of the code. This does not exclude bugs in the VM, but it does reduce the likelihood of malicious code, that is code explicitly engineered for an exploit.

When executing trusted code, the checks discussed in the previous section are disabled. Since checks like type safety or reference safety collect information during execution, care must be taken to correctly model the transitions from untrusted into trusted code and vice versa. For example, trusted code does not use the type stack, but untrusted code expects the type of a return value from a function call on the type stack, so this information needs to be synthesized when returning from a call to a trusted function into untrusted code.

5 Asynchronous Checking

Aptos executes blocks of transactions in parallel using *Block-STM* [7], an optimistic concurrency-control protocol in which a pool of worker threads speculatively runs transactions out of order and re-validates them against later-published writes; incarnations that lose a race are aborted and replayed. In practice most blocks do not saturate the worker pool, and aborted-and-restarted incarnations leave threads idle for non-trivial windows, so a substantial slice of worker capacity remains unused during normal operation.

We use that idle capacity to run the runtime safety checks *asynchronously*. During execution the interpreter trails a compact *execution trace* for each transaction: a count of successfully executed instructions, a fingerprint hash of the executed opcodes (used as an integrity tag), the bit-vector of conditional-branch outcomes, and the list of dynamically dispatched calls (closures and entry points; static calls are reconstructable from the bytecode and are not recorded). The trace contains no values, only control-flow shape, so it stays small.

When a transaction’s commit-time incarnation finalizes, an idle worker replays its trace by abstractly interpreting the bytecode along the recorded path — following the recorded branches, dispatching the recorded closures — and runs the type and ability checks of Sections 3.1 and 3.2 on the fully-substituted types. Because the trace records only *successful* instructions, gas need not be re-charged; the async checker uses an unmetered gas meter, and the recorded fingerprint catches any divergence between the original and the replayed instruc-

Table 1: Throughput (tx/s) under type and ability checking, slowdown vs the no-checks baseline. “Waiver” skips trusted framework code (Section 4); “async” defers checks to trace replay (Section 5).

workers	baseline	synchronous		asynchronous	
	(no checks)	all code	waiver	all code	waiver
2	864	611 (−29.3%)	739 (−14.5%)	513 (−40.6%)	624 (−27.7%)
4	1427	1026 (−28.1%)	1236 (−13.4%)	914 (−35.9%)	1088 (−23.7%)
8	1915	1350 (−29.5%)	1620 (−15.4%)	1427 (−25.5%)	1598 (−16.5%)
16	2019	1427 (−29.3%)	1691 (−16.2%)	1690 (−16.3%)	1799 (−10.9%)
32	1904	1363 (−28.4%)	1621 (−14.9%)	1633 (−14.2%)	1716 (−9.9%)

Table 2: Throughput (tx/s) under reference safety checking, all code, synchronous, in isolation (type and ability checks disabled).

workers	checks off	checks on	slowdown
2	858	630	−26.5%
4	1430	1054	−26.3%
8	1918	1370	−28.6%
16	2011	1450	−27.9%
32	1893	1375	−27.4%

tion streams. A violation found during replay aborts the block before its writes become observable to subsequent blocks.

6 Performance

We benchmark on a perpetual-futures DEX with 100 markets, driven by a weighted mix of oracle price updates, bulk market-maker orders, and retail taker orders, executed by Block-STM [7] on a 60-vCPU Google Cloud instance with 235 GB of RAM as the worker pool varies from 2 to 32 threads. Each reported number is the mean of three runs; run-to-run spread is below 1.3%.

Table 1 reports the cost of type and ability checking, which share a single per-instruction layer. Synchronously, the combined checks cost 28–29% of throughput, uniformly across worker counts. The trusted-code waiver of Section 4 roughly halves this to 13–16%, as about two thirds of executed instructions on this workload fall into trusted framework code. The asynchronous mode of Section 5 trades a small tracing overhead for deferred checking on idle workers: it is slower than synchronous below 8 workers, reaches parity at 8, and at 16–32 brings the cost to 10–11%, the preferred configuration at production concurrency.

Table 2 reports reference safety checking in isolation: 26–29%, again uniform across worker counts. The reference checker does not yet support the trusted-code waiver or asynchronous deferral; the type-check numbers above set a reasonable expectation for what extending these mechanisms should recover.

7 Conclusion

Runtime bytecode verification at module load time is standard for typed managed virtual machines: the Java Virtual Machine [12] and the Common Language Runtime [14] both run a verifier at class-load time to check well-formedness, typing, and stack discipline before bytecode is executed, and the sibling Move VMs deployed elsewhere — notably the original Diem MoveVM [4] and Sui’s variant — inherit Move’s bytecode verifier. To the best of our knowledge, none of these platforms maintains a *second* layer of runtime safety checks during execution: once the verifier admits a module, its dynamic semantics is trusted. At the opposite end of the spectrum, dynamically typed languages such as JavaScript or Python perform precisely the runtime type checks of Section 3.1, but lack any first, static layer. The Ethereum Virtual Machine [9, 17] and Solana’s SVM sit in a third regime: their bytecode is untyped (a stack of 256-bit words for the EVM; raw memory regions for the SVM), so neither static bytecode verification nor a runtime type discipline applies. Aptos’s combination of a static bytecode verifier with a complementary runtime layer is, to our knowledge, unique among production VMs and goes beyond the safety guarantees of any other smart-contract platform.

Regarding dynamic reference safety checks, to the best of our knowledge no one has attempted this before. Existing work on Rust-style borrow semantics [10, 11, 16] is described as a static analysis. The dynamic semantics is also interesting as a reference model for what reference safety means, as most static analyses are over-approximations determined by the particular method chosen, and a canonical semantics does not exist. More work is needed to fully formalize and categorize dynamic reference safety.

Runtime safety checks were added to the Move VM after its original design, which leads to many unnecessary inefficiencies. We are currently working on a new VM that treats dynamic checking and high performance as first principles.

Disclosure of Interests. All authors are employees of Aptos Labs and may hold equity in the company. The work described is implemented in the Aptos blockchain.

References

1. Amsden, Z., Arora, R., Bano, S., Baudet, M., Blackshear, S., Bothra, A., Cabrera, G., Catalini, C., Chalkias, K., Cheng, E., Ching, A., Chursin, A., Danezis, G., Giacomo, G.D., Dill, D.L., Ding, H., Doudchenko, N., Gao, V., Gao, Z., Garillot, F., Gorven, M., Hayes, P., Hou, J.M., Hu, Y., Hurley, K., Lewi, K., Li, C., Li, Z., Malkhi, D., Margulis, S., Maurer, B., Mohassel, P., de Naurois, L., Nikolaenko, V., Nowacki, T., Orlov, O., Perelman, D., Pott, A., Proctor, B., Qadeer, S., Rain, Russi, D., Schwab, B., Sezer, S., Sonnino, A., Venter, H., Wei, L., Wernerfelt, N., Williams, B., Wu, Q., Yan, X., Zakian, T., Zhou, R.: The Libra Blockchain. <https://developers.libra.org/docs/the-libra-blockchain-paper> (2019)
2. Aptos Labs: The Aptos Framework Book. <https://aptos-labs.github.io/framework-book/> (2023), accessed: 2026-05-09
3. Aptos Labs: The Move on Aptos Book. <https://aptos-labs.github.io/move-book/> (2023), accessed: 2026-05-09

4. Blackshear, S., Cheng, E., Dill, D.L., Gao, V., Maurer, B., Nowacki, T., Pott, A., Qadeer, S., Rain, Russi, D., Sezer, S., Zakian, T., Zho, R.: Move: A Language With Programmable Resources (2019), <https://developers.libra.org/docs/move-paper>
5. Blackshear, S., Dill, D.L., Qadeer, S., Barrett, C.W., Mitchell, J.C., Padon, O., Zohar, Y.: Resources: A safe language abstraction for money (2020), <https://arxiv.org/abs/2004.05106>
6. Dill, D.L., Grieskamp, W., Park, J., Qadeer, S., Xu, M., Zhong, J.E.: Fast and reliable formal verification of smart contracts with the move prover. In: Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2022). Lecture Notes in Computer Science, vol. 13243, pp. 183–200. Springer (2022). https://doi.org/10.1007/978-3-030-99524-9_10
7. Gelashvili, R., Spiegelman, A., Xiang, Z., Danezis, G., Li, Z., Malkhi, D., Xia, Y., Zhou, R.: Block-STM: Scaling blockchain execution by turning ordering curse to a performance blessing. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP 2023). pp. 232–244. ACM (2023). <https://doi.org/10.1145/3572848.3577524>
8. Grieskamp, W., Zhang, T., Kashyap, V., Silverman, J.: Formal verification of imperative first-class functions in Move. CoRR **abs/2605.10007** (2026), <https://arxiv.org/abs/2605.10007>
9. Hildenbrandt, E., Saxena, M., Rodrigues, N., Zhu, X., Daian, P., Guth, D., Moore, B.M., Park, D., Zhang, Y., Stefanescu, A., Rosu, G.: KEVM: A complete formal semantics of the ethereum virtual machine. In: CSF. pp. 204–217. IEEE Computer Society (2018)
10. Jung, R., Dang, H.H., Kang, J., Dreyer, D.: Stacked borrows: An aliasing model for Rust. Proc. ACM Program. Lang. **4**(POPL), 41:1–41:32 (2020)
11. Jung, R., Jourdan, J.H., Krebbers, R., Dreyer, D.: RustBelt: Securing the foundations of the Rust programming language. Proc. ACM Program. Lang. **2**(POPL), 66:1–66:34 (2018)
12. Lindholm, T., Yellin, F.: The Java Virtual Machine Specification. Addison-Wesley (1997)
13. Matsakis, N.D., Klock, H., F.S.: The Rust Language, `nourl = http://doi.acm.org/10.1145/2692956.2663188`. Ada Lett. **34**(3), 103–104 (Oct 2014). <https://doi.org/10.1145/2692956.2663188>
14. Meijer, E., Wa, R., Gough, J.: Technical overview of the common language runtime (2000)
15. Park, J., Zhang, T., Grieskamp, W., Xu, M., Giacomo, G.D., Chen, K., Lu, Y., Chen, R.: Securing Aptos Framework with Formal Verification. In: 5th International Workshop on Formal Methods for Blockchains (FMBC 2024). Open Access Series in Informatics (OASICS), vol. 118, pp. 9:1–9:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik (2024). <https://doi.org/10.4230/OASICS.FMBC.2024.9>
16. Villani, N., Dang, H.H., Jourdan, J.H., Jung, R., Dreyer, D.: Tree borrows. Proc. ACM Program. Lang. **9**(PLDI) (2025)
17. Wood, G.: Ethereum: A secure decentralised generalised transaction ledger (2014), <https://ethereum.github.io/yellowpaper/paper.pdf>